LABORATORIO DE SEGURIDAD — Django (Python)

Aseguramiento del aplicativo de la tienda · Desarrollo de Aplicaciones Web

Estudiante(s): _____ **Fecha:** _____

> Los nombres usados en la guía (`Usuario`, `Pedido`, `/api/productos`) son de ejemplo: adáptelos a los de su proyecto. Se prueba solo en el propio proyecto.

Parte 0 — Preparación y diagnóstico

1. Verifiquen que el proyecto **arranca** y que pueden usarlo antes de tocarlo.
2. Creen una rama de trabajo para poder volver atrás si algo se daña:

```
git checkout -b seguridad
```

3. Creen en la raíz del proyecto el archivo `INFORME_SEGURIDAD.md` con esta tabla (la irán llenando):

```
| # | Hallazgo | OWASP | Evidencia (cómo lo comprobé) | Corrección aplicada | Estado |
|---|----------|-------|------------------------------|---------------------|--------|
```

Diagnóstico inicial

Con la búsqueda global del editor (**Ctrl + Shift + F**) revisen cada punto y anoten **Sí / No / No aplica**. Cada «Sí» en una pregunta marcada con ⚠ es un hallazgo para el informe.

Pregunta	Qué buscar	Sí / No
⚠ ¿Hay contraseñas guardadas tal cual?	Campo `password`/`clave` que se guarda sin hash	
⚠ ¿Hay credenciales en el código?	`mongodb+srv`, `SECRET_KEY`, `password` escritos en `settings.py` o en las vistas	
⚠ ¿Hay archivos de secretos en git?	`git ls-files` muestra `.env` o similares	
⚠ ¿Se muestra texto de usuarios como HTML?	`innerHTML`, `|safe`, `mark_safe`, `{% autoescape off %}`, `v-html`, `dangerouslySetInnerHTML`	
⚠ ¿Se puede ver un recurso de otro usuario cambiando el id?	Rutas del tipo `/api/pedidos/:id` sin comprobar propietario	
⚠ ¿Las rutas de administrador solo están «ocultas» en el frontend?	Botones escondidos, pero el endpoint no verifica el rol	
⚠ ¿CORS está abierto a cualquier origen?	`CORS_ALLOW_ALL_ORIGINS = True` o `CORS_ORIGIN_ALLOW_ALL = True`	
⚠ ¿El proyecto usa HTTP simple?	Se abre con `http://`	

Debes ver… la tabla completa y el proyecto en la rama `seguridad`. Es normal que haya varios «Sí»: ese es el punto de partida.

Revisen también `settings.py`: ¿`DEBUG = True`? ¿`ALLOWED_HOSTS = ['*']`? ¿`SECRET_KEY` escrita en el archivo? Cada «sí» es un hallazgo **A02 – Configuración insegura**. Y ejecuten la herramienta de Django que audita la configuración:

```
python manage.py check --deploy
```

Anoten cuántas advertencias aparecen: al final del laboratorio deben ser muchas menos.

> **Nota:** esta guía usa el sistema de usuarios incluido en Django (`django.contrib.auth`). Si su proyecto guarda usuarios en MongoDB con otra librería (MongoEngine, Djongo), la lógica es la misma: cambie las consultas a su modelo y use `make_password` / `check_password` de `django.contrib.auth.hashers` para el hash.

Parte 1 — HTTPS local

Idea: con HTTPS el contenido viaja cifrado y el navegador comprueba el certificado. En desarrollo se usa un certificado local de **mkcert**, en el que el propio computador confía.

1.1 Generar el certificado

1. Instalen **mkcert** (https://github.com/FiloSottile/mkcert; en Windows con `choco install mkcert` o `scoop install mkcert`).
2. En la raíz del proyecto:

```
mkcert -install
mkdir certs
mkcert -key-file certs/key.pem -cert-file certs/cert.pem localhost 127.0.0.1
```

3. Agreguen al `.gitignore` los archivos generados: **la clave privada nunca se sube al repositorio.**

> Si no pueden instalar mkcert, usen OpenSSL (autofirmado): `openssl req -x509 -newkey rsa:2048 -nodes -keyout certs/key.pem -out certs/cert.pem -days 365 -subj "/CN=localhost"` El navegador mostrará una advertencia: es el comportamiento esperado de un certificado autofirmado (clase, Bloque 2).

1.2 Activar HTTPS en Django (Python)

El servidor de desarrollo normal (`runserver`) no habla HTTPS. Se usa `runserver_plus`, de *django-extensions*:

```
pip install django-extensions werkzeug pyOpenSSL
```

Agreguen `"django_extensions"` a `INSTALLED_APPS` y ejecuten:

```
python manage.py runserver_plus --cert-file certs/cert.pem --key-file certs/key.pem 8443
```

En `settings.py`, las banderas de HTTPS de Django:

```
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 300            # 5 min: SOLO para el laboratorio
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_REFERRER_POLICY = 'same-origin'
SECURE_CONTENT_TYPE_NOSNIFF = True
```

1.3 Comprobar

- Abran **https://localhost:8443** y verifiquen el **candado** en la barra de direcciones.
- Hagan clic en el candado → «La conexión es segura» → **certificado**: revisen el dominio, la vigencia y quién lo emitió.
- `SECURE_SSL_REDIRECT` redirige a HTTPS cualquier petición HTTP que llegue a Django. En desarrollo solo existe el puerto 8443 con HTTPS, así que la redirección se comprueba en producción; aquí basta verificar que la página carga con candado.

> **Ojo con HSTS en `localhost`.** La cabecera HSTS obliga al navegador a usar HTTPS con ese **nombre de host, en cualquier puerto**. Por eso en este laboratorio se usa una vigencia **corta (5 minutos)**. En producción se usan valores de un año (`31536000`). Si otro proyecto en `http://localhost:xxxx` empieza a redirigir a HTTPS, es esto: espere los 5 minutos o límpienlo en `chrome://net-internals/#hsts`.

Debes ver… la aplicación en `https://localhost:8443` con candado, y el certificado emitido por *mkcert*.

Pregunta: ¿qué dos garantías da HTTPS que HTTP no da? ¿Garantiza que el sitio sea honesto? (Respondan en el informe.)

Parte 2 — Registro y login: hash, token y límite de intentos

Idea: nunca se guarda la contraseña, solo su **hash lento con sal**. Al iniciar sesión se compara y se entrega un **token firmado con vencimiento**, guardado en una **cookie `HttpOnly`** (JavaScript no puede leerla: un XSS no se la lleva).

2.1 Instalar lo necesario

```
pip install pyjwt python-dotenv
```

Creen una app para la autenticación si aún no existe (`python manage.py startapp cuentas`), agréguenla a `INSTALLED_APPS` y definan en `settings.py`:

```
import os
JWT_SECRET = os.environ['JWT_SECRET']     # se carga del .env (Parte 6)
```

2.2 Registro con contraseña hasheada

Reglas que se aplican en el servidor (nunca solo en el frontend):

- El correo y la contraseña se **convierten a texto** y se validan (formato y largo mínimo de 10).
- El **rol nunca viene del cuerpo de la petición**: lo asigna el servidor (`cliente`). Si lo aceptaran del cliente, cualquiera se registraría como administrador.

```
# cuentas/views.py
import json, re
from django.contrib.auth import authenticate
from django.contrib.auth.models import User
from django.http import JsonResponse
from django.views.decorators.http import require_POST

def _datos(request):
    try:
        return json.loads(request.body or '{}')
    except ValueError:
        return {}

@require_POST
def registro(request):
    d = _datos(request)
    email = str(d.get('email', '')).strip().lower()
    clave = str(d.get('clave', ''))

    if not re.match(r'^\S+@\S+\.\S+$', email) or len(clave) < 10:
        return JsonResponse({'error': 'Datos inválidos'}, status=400)
    if User.objects.filter(username=email).exists():
        return JsonResponse({'error': 'Ese correo ya está registrado'}, status=409)

    # create_user guarda el hash (PBKDF2 por defecto), nunca la clave;
    # el rol lo decide el servidor: un usuario nuevo nunca es is_staff
    User.objects.create_user(username=email, email=email, password=clave)
    return JsonResponse({'ok': True}, status=201)
```

Django ya hace el hash por usted. En la base de datos el campo `password` debe empezar por `pbkdf2_sha256$`.

Prueba: registren un usuario y revisen en **Atlas** (o en la base de datos que usen) cómo quedó guardada la contraseña. Debe ser una cadena larga que empieza por `$2b$`, `$2a$`, `pbkdf2_sha256$` o similar, **nunca la clave original**.

2.3 Login que entrega el token

```
# cuentas/seguridad.py
import datetime, jwt
from django.conf import settings
from django.core.cache import cache

def rol_de(user):
    return 'admin' if user.is_staff else 'cliente'

def crear_token(user):
    ahora = datetime.datetime.now(datetime.timezone.utc)
    return jwt.encode(
        {'sub': str(user.pk), 'rol': rol_de(user), 'exp': ahora + datetime.timedelta(minutes=15)},
        settings.JWT_SECRET, algorithm='HS256')

def limitado(request, maximo=5, ventana=900):
    clave = 'login:' + request.META.get('REMOTE_ADDR', '?')
```

```
    cache.add(clave, 0, ventana)
    return cache.incr(clave) > maximo

# cuentas/views.py (continuación)
from django.views.decorators.csrf import ensure_csrf_cookie
from .seguridad import crear_token, rol_de, limitado

@ensure_csrf_cookie
def csrf(request):                      # el frontend la llama una vez al cargar
    return JsonResponse({'ok': True})

@require_POST
def login_view(request):
    if limitado(request):
        return JsonResponse({'error': 'Demasiados intentos, espere 15 minutos'}, status=429)
    d = _datos(request)
    user = authenticate(request,
                        username=str(d.get('email', '')).strip().lower(),
                        password=str(d.get('clave', '')))
    if user is None:
        return JsonResponse({'error': 'Usuario o contraseña incorrectos'}, status=401)

    resp = JsonResponse({'ok': True, 'rol': rol_de(user)})
    resp.set_cookie('token', crear_token(user), max_age=900,
                    httponly=True, secure=True, samesite='Strict')
    return resp

@require_POST
def logout_view(request):
    resp = JsonResponse({'ok': True})
    resp.delete_cookie('token')
    return resp
```

Rutas (`cuentas/urls.py`, incluido en `/api/auth/`): `registro/`, `login/`, `logout/` y `csrf/`.

Fíjense en tres decisiones:

- **Mensaje genérico:** «Usuario o contraseña incorrectos», sin decir cuál falló (evita enumerar correos).
- **Token corto:** vence en 15 minutos.
- **Cookie con tres banderas:** `HttpOnly` (JavaScript no la lee), `Secure` (solo por HTTPS) y `SameSite=Strict` (no se envía desde otros sitios).

2.4 Proteger rutas con el token

```
# cuentas/seguridad.py (continuación)
from functools import wraps
from django.http import JsonResponse

def autenticado(rol=None):
    def deco(vista):
        @wraps(vista)
        def interna(request, *args, **kwargs):
            try:
                datos = jwt.decode(request.COOKIES.get('token', ''),
                                   settings.JWT_SECRET, algorithms=['HS256'])
            except jwt.PyJWTError:
                return JsonResponse({'error': 'No autenticado'}, status=401)
            if rol and datos.get('rol') != rol:
                return JsonResponse({'error': 'Sin permiso'}, status=403)
            request.usuario_id = int(datos['sub'])
            request.rol = datos['rol']
            return vista(request, *args, **kwargs)
        return interna
```

```
    return deco
```

Uso: `@autenticado()` sobre una vista que solo requiere sesión.

Apliquen la protección a las rutas que **modifican** datos o que son privadas (crear/editar/borrar productos, ver pedidos). Mantengan públicas las que son de consulta, como el catálogo.

2.5 Limitar intentos de login

Ya quedó en `limitado()` (Parte 2.3): cuenta los intentos por IP en la caché de Django. Con la caché por defecto en memoria funciona en el laboratorio (en producción se usa Redis).

Prueba: intenten iniciar sesión 6 veces seguidas con una clave incorrecta. El sexto intento debe responder **429**.

Debes ver…

- La contraseña en la base de datos como hash.
- `POST` al login con datos correctos → **200** y una cookie `token` en *F12 → Application → Cookies* con `HttpOnly` y `Secure` marcadas.
- Una ruta privada sin sesión → **401**.
- El sexto intento fallido → **429**.

Prueba de que HttpOnly funciona: en la consola del navegador escriban `document.cookie`. La cookie `token` **no debe aparecer**.

Parte 3 — Roles y control de acceso

Idea: autenticarse no basta. Hay que decidir **qué puede hacer cada rol**, siempre **en el servidor**: esconder un botón en el frontend no es seguridad.

Rol	Ver catálogo	Comprar	Crear/editar/borrar productos	Ver todos los pedidos
Visitante	Sí	No	No	No
Cliente	Sí	Sí	No	Solo los suyos
Administrador	Sí	Sí	Sí	Sí

3.1 Proteger las rutas de administración por rol

```
from cuentas.seguridad import autenticado

@autenticado(rol='admin')       # solo administradores
@require_http_methods(['POST'])
def crear_producto(request):
    ...

@autenticado(rol='admin')
@require_http_methods(['PUT', 'DELETE'])
def editar_o_borrar_producto(request, pk):
    ...
```

Cómo crear un administrador para probar: cambien el rol directamente en la base de datos (Atlas), no con un endpoint público.

Con la terminal de Django:

```
python manage.py shell

from django.contrib.auth.models import User
u = User.objects.get(username='correo@dominio.com')
u.is_staff = True        # en esta guía, is_staff = administrador
u.save()
```

3.2 Arreglar el IDOR (ver el recurso de otro)

1. Con **dos usuarios** distintos (A y B), creen un pedido con A y anoten su `id`.
2. Inicien sesión como B y pidan `GET /api/pedidos/<id de A>`. **Antes de corregir**, si ven el pedido de A, tienen un hallazgo **A01 – Control de acceso roto** para el informe.
3. Corrijan para que la consulta incluya siempre al propietario:

```
        from django.shortcuts import get_object_or_404

        @autenticado()
        def pedido_detalle(request, pk):
            if request.rol == 'admin':
                pedido = get_object_or_404(Pedido, pk=pk)
            else:
                pedido = get_object_or_404(Pedido, pk=pk, cliente_id=request.usuario_id)  # solo los suyos
            return JsonResponse({'id': pedido.pk, 'total': pedido.total})

        @autenticado()
        def pedidos_lista(request):
            qs = Pedido.objects.all() if request.rol == 'admin' else
Pedido.objects.filter(cliente_id=request.usuario_id)
            return JsonResponse({'pedidos': list(qs.values('id', 'total'))})
```

Debes ver…

- Un cliente que intenta `DELETE /api/productos/<id>` → **403**.
- Un administrador que hace lo mismo → **200/204**.
- B pidiendo el pedido de A → **404** (no se revela ni que existe).

Añadan los hallazgos al informe.

Parte 4 — XSS: demostrar, corregir y reforzar

4.1 Demostrar el ataque (en su propio proyecto)

1. Busquen en el frontend dónde se muestran textos escritos por usuarios (nombre, reseñas, comentarios, descripciones de producto):

- En JavaScript: `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `document.write`.
- En plantillas de Django: el filtro `|safe`, la función `mark_safe` en las vistas y el bloque `{% autoescape off %}`.
- En React: `dangerouslySetInnerHTML`; en Vue: `v-html`.

2. Si hay un campo de texto libre que se muestra así, publiquen este texto de prueba:

```
        <img src=x onerror="alert('XSS')">
```

3. Si aparece la alerta, **hay un XSS almacenado**: anótenlo (**A05 – Inyección**, con captura de pantalla como evidencia).

> Si su proyecto no tiene ningún campo de texto libre, prueben en el muro de ideas del taller o creen un campo de «reseña» para el ejercicio.

4.2 Corregir

Regla: todo dato de usuario es **texto**, no HTML.

Antes (vulnerable)	Después (seguro)
`tarjeta.innerHTML = '<p>' + texto + '</p>'`	`const p = document.createElement('p'); p.textContent = texto; tarjeta.append(p);`
`{{ producto.descripcion|safe }}` (Django)	`{{ producto.descripcion }}`
`mark_safe(texto_del_usuario)`	Quitar `mark_safe`: Django escapa solo
`<div v-html="resena">` (Vue)	`<div>{{ resena }}</div>`

Las plantillas de Django **escapan por defecto**: el peligro está en desactivar esa protección. Si **de verdad** hay que permitir HTML (editor enriquecido), se limpia con una librería probada como **nh3** o **bleach** en el servidor.

Repitan la prueba: ahora debe verse el texto literal `<img src=x ...>` en pantalla, sin alerta.

4.3 Reforzar con cabeceras de seguridad y CSP

Aunque se corrija el código, se añade otra capa: la **Content-Security-Policy** le dice al navegador qué scripts puede ejecutar.

Django ya envía `X-Content-Type-Options` y `X-Frame-Options`. Para la CSP se añade un middleware sencillo:

```
# cuentas/middleware.py
class CSPMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        r = self.get_response(request)
        r['Content-Security-Policy'] = (
            "default-src 'self'; script-src 'self'; "
            "style-src 'self' 'unsafe-inline'; img-src 'self' data:")
        return r

# settings.py → MIDDLEWARE: añadir  'cuentas.middleware.CSPMiddleware'  al final de la lista
```

(Existe también la librería `django-csp`; para el laboratorio basta el middleware.)

> **Si algo se rompe:** la CSP bloquea los *scripts en línea* (`<script>...</script>` dentro del HTML) y los manejadores como `onclick="..."`. Muévanlos a archivos `.js` aparte. Si usan una librería por CDN (Bootstrap, Tailwind CDN, etc.), agreguen su dominio a la política. Revisen *F12 → Console*: el navegador dice qué bloqueó.

Prueba de la defensa en profundidad: vuelvan **temporalmente** a poner `innerHTML` con el texto de prueba. Con la CSP activa, el `onerror` en línea **debe quedar bloqueado** y verán el error en la consola. Después **restauren** la corrección.

Debes ver…

- El texto de prueba mostrado como texto, sin alerta.
- En *F12 → Network*, en la respuesta de cualquier página: la cabecera `Content-Security-Policy` y las demás (`X-Content-Type-Options`, etc.).

Parte 5 — CSRF

Idea: como la sesión va en una **cookie**, el navegador la envía solo, incluso desde una página maliciosa. Se defiende con (1) `SameSite` en la cookie, que ya pusieron, y (2) un **token anti-CSRF** que solo la página legítima conoce.

Estrategia de este laboratorio (doble envío de cookie): el servidor entrega una cookie `csrftoken` legible por JavaScript; el frontend la copia en la cabecera `X-CSRFToken` de cada petición que **modifica** datos (`POST`, `PUT`, `PATCH`, `DELETE`); el servidor comprueba que coincidan. Una página de otro sitio no puede leer la cookie ni ponerla en la cabecera.

5.1 Servidor

Django trae protección CSRF activada por defecto (`CsrfViewMiddleware`). Verifiquen en `settings.py` que **sigue** en `MIDDLEWARE` y que **ninguna vista** que cambie datos tenga `@csrf_exempt` (búsquenlo con Ctrl + Shift + F). Añadan:

```
CSRF_COOKIE_SAMESITE = 'Strict'
CSRF_COOKIE_HTTPONLY = False                # el frontend debe poder leerla (es lo normal con fetch)
CSRF_TRUSTED_ORIGINS = ['https://localhost:8443']
```

El frontend debe llamar **una vez** a `GET /api/auth/csrf/` al cargar la página (eso crea la cookie `csrftoken`) y luego enviarla en la cabecera `X-CSRFToken` en cada petición que modifique datos, **incluidos el registro y el login**.

Si no usan `@csrf_exempt`, **no hay más código que escribir**. Respondan en el informe: ¿por qué alguien pondría `@csrf_exempt` «porque da error 403» y por qué es mala idea?

5.2 Frontend

El frontend lee la cookie y la manda en la cabecera. Con `fetch`:

```
function leerCookie(nombre) {
  const par = document.cookie.split('; ').find(c => c.startsWith(nombre + '='));
  return par ? decodeURIComponent(par.split('=')[1]) : '';
```

```
}

await fetch('/api/productos', {
  method: 'POST',
  credentials: 'include',   // envía la cookie de sesión
  headers: {
    'Content-Type': 'application/json',
    'X-CSRFToken': leerCookie('csrftoken')
  },
  body: JSON.stringify(producto)
});
```

Hagan lo mismo en **todas** las llamadas que modifiquen datos. Si usan `axios`, se puede configurar una sola vez en un interceptor; con React/Vue/Angular se centraliza en el servicio que hace las peticiones.

5.3 Comprobar

- Creen un archivo **fuera del proyecto**, `ataque.html`, con este formulario (cambien la URL y los campos a los de su ruta):

```
<form action="https://localhost:8443/api/productos" method="POST">
  <input name="nombre" value="producto-del-atacante">
</form>
<script>document.forms[0].submit()</script>
```

- Con la sesión de administrador abierta en el navegador, ábranlo (doble clic). **No debe crearse nada.** El servidor responde **403** (o el navegador no envía la cookie por `SameSite`).
- Compruebe también que desde su frontend **sí** funciona crear un producto.

Debes ver… en *F12 → Network* que sus peticiones normales llevan la cabecera `X-CSRFToken` y funcionan, y que la petición falsificada se rechaza.

Pregunta: si ya usan `SameSite=Strict`, ¿por qué se añade además el token? (Respondan en el informe: defensa en profundidad, navegadores antiguos, subdominios.)

Parte 6 — Secretos, CORS, errores y dependencias

6.1 Secretos fuera del código

1. Todo lo secreto va en un archivo `.env` (cadena de Atlas, secreto del token, contraseña del certificado) y **`.env` va en el `.gitignore`**. Creen además `.env.example` **sin valores reales** para documentar qué variables hacen falta.
2. Genere un secreto largo y aleatorio para el token:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

3. Verifiquen que **no hay secretos ya subidos**:

```
git log --all -p -S"mongodb+srv" | head -50
```

> **Si aparece una credencial real en el historial de git, el repositorio ya la filtró.** No basta con borrarla: hay que **rotarla** (cambiar la contraseña del usuario de Atlas) y recién después limpiar. Es el caso de Uber 2016 visto en clase. Anótenlo como hallazgo **A04/A02**.

```
# settings.py (arriba)
import os
from dotenv import load_dotenv
load_dotenv()

SECRET_KEY = os.environ['DJANGO_SECRET_KEY']      # la de Django, distinta a JWT_SECRET
JWT_SECRET = os.environ['JWT_SECRET']
DEBUG = os.environ.get('DEBUG', '0') == '1'
ALLOWED_HOSTS = ['localhost', '127.0.0.1']

# conexión a Atlas
MONGO_URI = os.environ['MONGO_URI']
```

Archivo `.env` (no se sube al repositorio):

```
DJANGO_SECRET_KEY=pegue-aqui-un-valor-generado
JWT_SECRET=pegue-aqui-otro-valor-generado
MONGO_URI=mongodb+srv://usuario:clave@cluster.mongodb.net/tienda
DEBUG=1
```

Importante: la `SECRET_KEY` que venía en `settings.py` ya quedó expuesta si estuvo en el repositorio: genere una nueva (el mismo comando de arriba).

6.2 Configuración de Atlas

En el panel de Atlas, revisen y anoten:

- **Network Access:** que **no** esté `0.0.0.0/0` (cualquier IP). Dejen solo su IP.
- **Database Access:** que el usuario de la aplicación tenga el permiso mínimo (`readWrite` solo sobre la base de la tienda), **no** `atlasAdmin`.

6.3 CORS con orígenes específicos

Si el frontend se sirve desde Django (misma URL), **no hace falta CORS**. Si el frontend está en otro origen:

```
pip install django-cors-headers

# settings.py
INSTALLED_APPS += ['corsheaders']
MIDDLEWARE.insert(0, 'corsheaders.middleware.CorsMiddleware')
CORS_ALLOWED_ORIGINS = ['https://localhost:5173']   # solo el origen de su frontend
CORS_ALLOW_CREDENTIALS = True
```

6.4 Errores genéricos hacia afuera, detalle en los registros

```
# settings.py
LOGGING = {
    'version': 1,
    'handlers': {'consola': {'class': 'logging.StreamHandler'}},
    'loggers': {'django': {'handlers': ['consola'], 'level': 'INFO'}},
}

# urls.py del proyecto: respuesta genérica cuando DEBUG está apagado
from django.http import JsonResponse
def error_500(request):
    return JsonResponse({'error': 'Error interno'}, status=500)
def error_404(request, exception):
    return JsonResponse({'error': 'No encontrado'}, status=404)
handler500 = error_500
handler404 = error_404
```

Con `DEBUG = True` Django muestra una página con código, rutas y variables a cualquiera. Para la prueba, pónganlo en `0` en el `.env`. (Con `DEBUG=0` los archivos estáticos no se sirven con `runserver`: para esta comprobación basta probar las rutas de la API, o usar *WhiteNoise*.)

Prueba: provoquen un error (por ejemplo, pidan un recurso con un id mal formado). La respuesta debe ser **un mensaje genérico**, sin rutas de archivos ni trazas; el detalle se ve en la terminal del servidor.

6.5 Dependencias

```
pip install pip-audit
pip-audit                       # revisa lo instalado en el entorno virtual
python manage.py check --deploy # vuelvan a correr el diagnóstico de la Parte 0
```

Compare el número de advertencias de `check --deploy` con el del inicio.

Anoten cuántas vulnerabilidades de nivel **alto o crítico** aparecen y qué paquete las trae.

Debes ver… el `.env` ignorado por git, el servidor fallando con un mensaje claro si falta `JWT_SECRET`, y la salida de la auditoría de dependencias.

Parte 7 — Auditoría OWASP e informe

Revisen el proyecto contra el OWASP Top 10 (2025) y completen el informe.

OWASP	Pregunta de revisión	Dónde se comprobó
A01 Control de acceso roto	¿Cada ruta verifica rol **y** propietario en el servidor?	Parte 3
A02 Configuración insegura	¿CORS, DEBUG, Atlas y cabeceras están bien?	Partes 4 y 6
A03 Cadena de suministro	¿Se auditaron las dependencias?	Parte 6.5
A04 Fallas criptográficas	¿HTTPS activo? ¿Contraseñas con hash lento? ¿Secretos fuera del código?	Partes 1, 2 y 6
A05 Inyección (incluye XSS)	¿Salida escapada y entradas validadas por tipo?	Partes 2 y 4
A06 Diseño inseguro	¿Hay lógica que se pueda abusar (cantidades negativas, cupones sin límite)?	Pendiente: pruébenlo
A07 Fallas de autenticación	¿Límite de intentos, mensajes genéricos, tokens con vencimiento?	Parte 2
A08 Integridad de software o datos	¿El token está firmado y se verifica? ¿Se confía en datos del cliente (precio, rol)?	Partes 2 y 3
A09 Registros y alertas	¿Se registran los intentos de login fallidos y los errores?	Parte 6.4
A10 Condiciones excepcionales	¿Los errores fallan de forma **segura** (niegan, no permiten)?	Parte 6.4

7.1 Dos pruebas de diseño inseguro (A06) para hacer ahora

1. **Cantidad negativa:** envíen una compra con `cantidad: -5` o un precio manipulado en el cuerpo de la petición. Si el servidor lo acepta, el total se vuelve negativo: **nunca se confía en el precio del cliente**; se lee de la base de datos.
2. **Campo de más:** en el registro, envíen `"rol": "admin"` en el cuerpo. El usuario creado debe seguir siendo `cliente`.

7.2 Completar el informe

Para **cada hallazgo** de la Parte 0 y de las pruebas, una fila de la tabla: qué era, a qué categoría OWASP pertenece, cómo lo comprobaron, qué hicieron y su estado (corregido, mitigado o pendiente).

Después, agreguen al final del informe:

1. **Matriz de riesgos:** 5 riesgos que **siguen** presentes en su proyecto, con probabilidad (baja/media/alta), impacto (bajo/medio/alto) y tratamiento (mitigar, aceptar, transferir o evitar).
2. **Plan ante un incidente:** si la cadena de conexión de Atlas se filtrara hoy, ¿qué pasos seguirían, en el orden del ciclo de NIST (preparación, detección, contención, erradicación, recuperación, lecciones aprendidas)?
3. **Datos personales:** ¿qué datos de clientes guarda la tienda y para qué? Según la Ley 1581 de 2012, ¿qué sobra y debería dejar de recolectarse?

7.3 Entrega

- Rama `seguridad` con **commits pequeños y claros** (uno por parte, por ejemplo `Parte 2: hash y login`).
- `INFORME_SEGURIDAD.md` completo.
- **Sin** `.env`, certificados ni claves privadas en el repositorio.

```
git add .
git commit -m "Parte N: descripción"
```

Si algo no funciona

Síntoma	Causa probable	Qué hacer
`KeyError: 'JWT_SECRET'` (o `DJANGO_SECRET_KEY`)	El `.env` no se cargó o falta la variable	Verificar `load_dotenv()` al inicio de `settings.py` y el archivo `.env`
`403 CSRF verification failed` en el login	El frontend no llamó antes a `/api/auth/csrf/` o no envía la cabecera `X-CSRFToken`	Llamar al endpoint al cargar y enviar la cabecera
`No module named 'jwt'`	Se instaló `jwt` en vez de `pyjwt`, o el `venv` no está activo	`pip uninstall jwt`, `pip install pyjwt`; ver `(venv)` en la terminal
`runserver_plus` no existe	Falta `django_extensions` en `INSTALLED_APPS` o `werkzeug`	Añadirlo e instalar `werkzeug`
`DisallowedHost`	`ALLOWED_HOSTS` no incluye `localhost`	Añadir `'localhost'` y `'127.0.0.1'`
`ERR_CERT_AUTHORITY_INVALID`	No se corrió `mkcert -install`	Ejecutarlo y reiniciar el navegador
Todo funcionaba y tras la Parte 4 el frontend se ve roto	La CSP bloquea scripts o estilos en línea o de un CDN	Abrir *F12 → Console*, leer qué se bloqueó y moverlo a un archivo o añadir el dominio a la política
El navegador no envía la cookie del token	Frontend en otro origen sin `credentials: 'include'`, o la página está en `http://` y la cookie es `Secure`	Servir todo por HTTPS y añadir `credentials: 'include'` en las peticiones
Un `POST` que antes funcionaba da 403 tras la Parte 5	Falta la cabecera anti-CSRF en esa llamada	Añadirla en **todas** las peticiones que modifican datos
Se bloqueó el login tras probar el límite	Es el límite de intentos funcionando	Esperar 15 minutos o reiniciar el servidor

Cierre

Respondan en el informe, en dos o tres líneas cada una:

1. ¿Cuál fue el hallazgo más grave de su proyecto y por qué?
2. ¿Qué defensa creen que habría evitado el caso de la clase que más les impactó (Equifax, Uber, British Airways, etc.)?
3. ¿Qué dejarían como mejora futura (MFA, renovación de tokens, registros centralizados, pruebas automáticas de seguridad)?

Recuerden: ninguna medida alcanza sola. La seguridad es **defensa en profundidad** y se piensa desde el diseño.